

Problem 1.2

April 19, 2026

0.1 Problem 1.2 — LQR as a QP

0.1.1 a)

From $x_{t+1} = Ax_t + Bu_t$, expanding recursively:

$$x_1 = Ax_0 + Bu_0$$

$$x_2 = A(Ax_0 + Bu_0) + Bu_1 = A^2x_0 + ABu_0 + Bu_1$$

By recursion we can write

$$x_t = A^t x_0 + \sum_{i=0}^{t-1} A^{t-1-i} B u_i$$

Writing $\mathbf{x} = (x_0, x_1, \dots, x_T)^\top$ and $u = (u_0, u_1, \dots, u_{T-1})^\top$, we have

$$\mathbf{x} = Mx_0 + Gu$$

where

$$M = \begin{pmatrix} I \\ A \\ A^2 \\ \vdots \\ A^T \end{pmatrix}, \quad G = \begin{pmatrix} 0 & 0 & \dots & 0 \\ B & 0 & \dots & 0 \\ AB & B & \dots & 0 \\ \vdots & & \ddots & \vdots \\ A^{T-1}B & A^{T-2}B & \dots & B \end{pmatrix}.$$

We note $H = \text{blkdiag}(Q, Q, \dots, Q, Q_T)$ and $L = \text{blkdiag}(R, \dots, R)$, so that

$$J(u) = \mathbf{x}^\top H \mathbf{x} + u^\top L u.$$

Substituting $\mathbf{x} = Mx_0 + Gu$:

$$\begin{aligned} J(u) &= (Mx_0 + Gu)^\top H (Mx_0 + Gu) + u^\top L u \\ &= x_0^\top M^\top H M x_0 + x_0^\top M^\top H G u + u^\top G^\top H M x_0 + u^\top G^\top H G u + u^\top L u \end{aligned}$$

The two cross terms are scalars, so $x_0^\top M^\top H G u = (x_0^\top M^\top H G u)^\top = u^\top G^\top H^\top M x_0$, and they combine into $2(G^\top H^\top M x_0)^\top u$:

$$J(u) = x_0^\top M^\top H M x_0 + 2(G^\top H^\top M x_0)^\top u + u^\top (G^\top H G + L) u.$$

This is exactly $\frac{1}{2} u^\top \bar{Q} u - \bar{b}^\top u + \text{const}$, so matching:

$$\boxed{\bar{Q} = 2(G^\top H G + L)}, \quad \boxed{\bar{b} = -2 G^\top H^\top M x_0}.$$

$\bar{Q}$ is positive-definite since $L \succ 0 \implies G^\top H G + L \succ 0$, so the QP has a unique minimiser $u^* = \bar{Q}^{-1} \bar{b}$.

0.1.2 b)

The QP from (a) is $\min_u \frac{1}{2} u^\top \bar{Q} u - \bar{b}^\top u$ with $\bar{Q} \succ 0$, so exactly the same setup as Problem 1.1. By 1.1(b), Newton's method converges in one step:

$$u^* = \bar{Q}^{-1} \bar{b}.$$

The optimal cost is then

$$J(u^*) = \frac{1}{2} u^{*\top} \bar{Q} u^* - \bar{b}^\top u^* + x_0^\top M^\top H M x_0.$$

Please see code.

```
[1]: import numpy as np
from scipy.linalg import block_diag

# Given data
n, m, T = 2, 1, 20
A = np.array([[1, 1], [0, 1]], dtype=float)
B = np.array([[0], [1]], dtype=float)
x0 = np.array([[1], [0]], dtype=float)
QT = 10 * np.eye(n)
Q = np.eye(n)
R = np.eye(m)

# Build M and G
M = np.vstack([np.linalg.matrix_power(A, t) for t in range(T + 1)])

G = np.zeros((n * (T + 1), m * T))
for t in range(1, T + 1):
    for i in range(t):
        G[t*n:(t+1)*n, i*m:(i+1)*m] = np.linalg.matrix_power(A, t - 1 - i) @ B

# Build H and L
H = block_diag(*([Q] * T + [QT]))
```

```

L = block_diag(*([R] * T))

# QP matrices
Qbar = 2 * (G.T @ H @ G + L)
bbar = -2 * G.T @ H.T @ M @ x0

# Newton: one step
u_star = np.linalg.solve(Qbar, bbar)

# Optimal cost:  $J(u^*) = 1/2 u^{*T} Qbar u^* - bbar^T u^* + x0^T M^T H M x0$ 
const = (x0.T @ M.T @ H @ M @ x0).item()
J_star = (0.5 * u_star.T @ Qbar @ u_star - bbar.T @ u_star).item() + const

print(f"J(u*) = {J_star:.6f}")

```

J(u*) = 2.947123